WNDRVR
SUPPORT NETWORK

VXWORKS 7 REAL-TIME NETWORK STACK PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Real-Time Network Stack Programmer's Guide, SR0650

10 June 2020

1. REAL-TIME NETWORK STACK OVERVIEW

1. Real-Time Network Stack Overview

The VxWorks® 7 Real-Time Network Stack Programmer's Guide describes how the VxWorks real-time network (RTNET) stack can be run within a VxWorks real-time process (RTP) or the VxWorks kernel. Running the stack in an RTP allows you to run multiple stack instances that are independent from both the VxWorks kernel and each other.

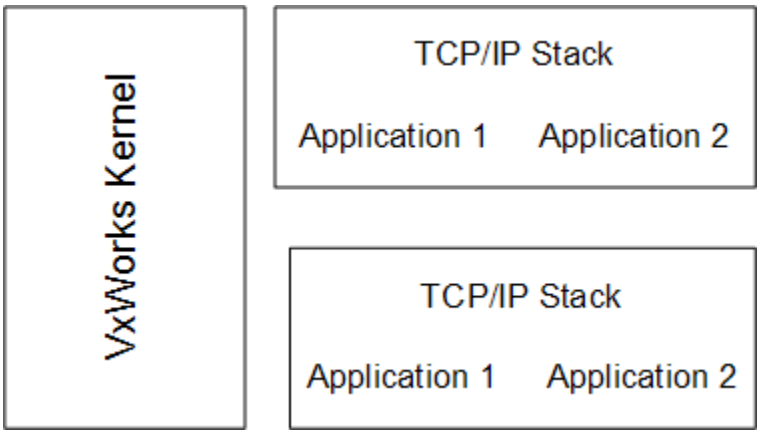
When the RTNET stack is executing inside an RTP, it is separated from the VxWorks kernel. This allows you to support multiple concurrently running instances that are totally isolated and independent from each other both in the space domain (separate RTPs and therefore separate address spaces) and the time domain (through the VxWorks time partition scheduler).

The RTNET stack is a host stack (it does not support acting as a router). The stack is designed to provide low latency, and more importantly, predictable latency on any operation. This is in contrast to a network stack implementation that is optimized for maximum throughput and, in some cases, for complex configurations such as a large forward information base (FIB). RTNET provides good throughput. However, it requires that the FIB be small; limited to entries for the default gateway, the locally attached networks, and a few additional entries as needed. This means entries should be limited to low tens, not tens of thousands, or even the millions of FIB entries that are possible in the standard VxWorks network stack (IPNET).

When the RTNET stack is run in an RTP, the entire network stack, including the driver layer, is contained within a single RTP to ensure isolation from other parts of the system. This helps achieve predictability. It also requires that to run multiple instances of the network stack, your target must have at least one network interface card per network stack instance. Requiring each stack instance to have a dedicated network interface avoids the timing dependencies that could be introduced if you were attempting to use shared hardware.

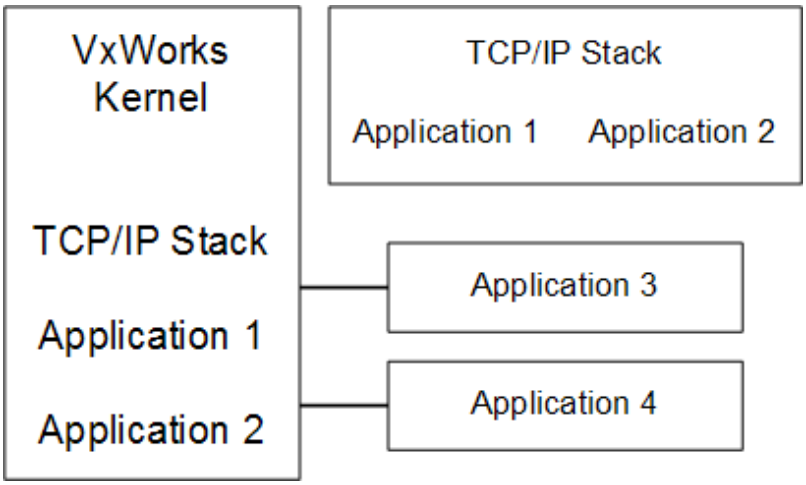
Any applications that use an instance of the TCP/IP network stack must reside within the same RTP as the network stack. You can include multiple applications in the RTP with the network stack instance.

The logical view of a system using this network stack is similar to the following, where each box represents a separate address space:



Each network stack instance contains all of the data structures required for protocol processing. That is, each network stack instance includes its own FIB, neighbor cache (ARP table), socket database, network interface structures, and so forth.

The RTNET stack also supports the more traditional kernel mode where the network stack resides in the VxWorks kernel and applications reside in either the kernel or in RTPs using system calls for BSD socket calls. A feature of the RTNET stack is that it supports stack instances running in RTPs at the same time with a stack instance running in the kernel at the same time. The logical view in this use case is similar to the following (where each box represent a separate address space):



Moving the network stack and application to user space implies that the BSD socket library also runs in user space. Thus, when used in RTPs, the RTNET stack depends on the user mode I/O system and user mode sockets features in VxWorks.

Related information

User Mode I/O System on page 2
Supported Protocols on page 3
Supported Socket Types on page 3
Supported RFCs on page 4
Supported BSPs and Drivers on page 21
Real-Time Network Stack Limitations on page 22

2. User Mode I/O System

The VxWorks 7 user mode I/O system (UMIOS) supports running the real-time network (RTNET) stack, together with a socket application, in a real-time process (RTP).

When using the traditional VxWorks network stack (IPNET), and a conventional socket application running in an RTP, the familiar sockets APIs are provided as system calls, and are implemented in the kernel jointly by the **sockLib** library and the network stack. The IPNET network stack provides a **sockLib** back end which handles sockets of the particular address families that the stack supports (such as **AF_INET**, **AF_INET6**, **AF_ROUTE**, **AF_NETLINK**, and so forth).

In this situation, **sockLib** plugs into the VxWorks I/O system as a (specialized) I/O system driver. Although sockets do not appear as named files or devices in the VxWorks I/O system, sockets are represented as file descriptors and you can use standard I/O APIs such as read( ), write( ), ioctl( ), or close( ) on these descriptors.

The primary feature of the RTNET stack is that you are able to run an instance of the RTNET stack in an RTP. When this is done, the RTNET network stack, **sockLib**, and the user application are all implemented in user mode in the RTP. Because **sockLib** is still implemented as an I/O system driver, part of the VxWorks I/O system (what is required to support **sockLib**) is implemented inside the RTP.

Note: In order to access a physical network from the user mode RTNET stack, you must also have a network device driver (an END driver) capable of running in an RTP. This type of network device driver interfaces with the MUX and with VxBus, but does not directly interface with the VxWorks I/O system. For information on user mode (RTP-based) drivers, see the VxWorks 7 BSP and Driver Guide. For more information on network device drivers, see the VxWorks 7 Network Driver Developer's Guide.

For more information about the user-mode I/O system, see the *VxWorks 7 File Systems and I/O Programmer's Guide*.

Related information
Real-Time Network Stack Overview on page 1
Supported Protocols on page 3
Supported Socket Types on page 3
Supported RFCs on page 4
Supported BSPs and Drivers on page 21
Real-Time Network Stack Limitations on page 22

3. Supported Protocols

The real-time network (RTNET) stack supports a number of standard networking protocols.

The RTNET stack currently supports the following protocols:

- Internet Protocol, version 4 (IPv4)
- Internet Protocol, version 6 (IPv6)
- Neighbor Discovery Protocol, version 6 (IPv6)
- Transmission Control Protocol (TCP)
- User Datagram Protocol (UDP)
- Address Resolution Protocol (ARP) for IPv4-to-Ethernet
- Internet Control Message Protocol (ICMP); no deprecated message types are implemented
- Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6); partly supported
- Internet Group Management Protocol (IGMP), version 2
- Internet Group Management Protocol (IGMP), version 3

Related information
Real-Time Network Stack Overview on page 1
User Mode I/O System on page 2
Supported Socket Types on page 3
Supported RFCs on page 4
Supported BSPs and Drivers on page 21
Real-Time Network Stack Limitations on page 22

4. Supported Socket Types

When using the real-time network (RTNET) stack, you can create several domains and socket types.

The following domains and socket types are available:

- **AF_INET**, **SOCK_STREAM** to use TCP over IPv4
- **AF_INET**, **SOCK_DGRAM** to use UDP over IPv4
- **AF_INET**, **SOCK_RAW**, *IPPROTO* to use IPv4 with application processing

Note: The **AF_NETLINK**, **SOCK_RAW**, **NETLINK_ROUTE** protocol to use netlink is also available. However, you should not use this protocol directly in your applications. Rather, a high-level API that uses netlink to enumerate ARP entries, sockets, and so forth is provided by VxWorks using **netViewLib**.

If you want to use the services implemented by this TCP/IP network stack in your application, you should develop your application using the VxWorks **sockLib** library as you would with any other application.

Related information

Real-Time Network Stack Overview on page 1
User Mode I/O System on page 2
Supported Protocols on page 3
Supported RFCs on page 4
Supported BSPs and Drivers on page 21
Real-Time Network Stack Limitations on page 22

5. Supported RFCs

The real-time network (RTNET) stack is compliant with the RFCs listed below. However, there are some limitations.

- Because RTNET is a host-only TCP/IP network stack, the portions of the RFCs that apply to a routing stack have not been implemented.
- Some portions of RFC 6528 have not been implemented because they require hardware random number generation.
- Not all of the requirements for RFC1122 have been implemented. The exceptions are described in RFC 1122 Conformance Details on page 5.

RFC	Title
0768	*User Datagram Protocol*
0791	*Internet Protocol*
0792	*Internet Control Message Protocol*
0793	*Transmission Control Protocol*
0826	*An Ethernet Address Resolution Protocol*
0879	*The TCP Maximum Segment Size and Related Topics*
0894	*A Standard for the Transmission of IP Datagrams over Ethernet Networks*
1071	*Computing the Internet Checksum*
1112	*Host Extensions for IP Multicasting*. Partial conformance, see RFC 1122 Conformance Details on page 5.
1191	*Path MTU Discovery*
1122	*Requirements for Internet Hosts -- Communication Layers*

RFC	Title
2236	Internet Group Management Protocol, Version 2
2464	*Transmission of IPv6 Packets over Ethernet Networks*
3376	Internet Group Management Protocol, Version 3
3390	*Increasing TCP's Initial Window*
3549	*Linux Netlink as an IP Services Protocol*
4861	*Neighbor Discovery for IP version 6 (IPv6)*
5681	*TCP Congestion Control*
5952	*A Recommendation for IPv6 Address Text Representation*
6298	*Computing TCP's Retransmission Timer*
6528	*Defending against Sequence Number Attacks*
6582	*The NewReno Modification to TCP's Fast Recovery Algorithm*
6633	*Deprecation of ICMP Source Quench Messages*
6864	*Updated Specification of the IPv4 ID Field*
6918	*Formally Deprecating Some ICMPv4 Message Types*
7323	*TCP Extensions for High Performance*
8200	*Internet Protocol, Version 6 (IPv6) Specification*
4861	*Neighbor Discovery for IP version 6 (IPv6)*

RFC 1122 Conformance Details on page 5

Related information

Real-Time Network Stack Overview on page 1
User Mode I/O System on page 2
Supported Protocols on page 3
Supported Socket Types on page 3
Supported BSPs and Drivers on page 21
Real-Time Network Stack Limitations on page 22

5.1. RFC 1122 Conformance Details

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Trailer encapsulation	2.3.1			x			N	Driver decides
Send Trailers by default without negotiation	2.3.1					x	N	Driver decides
ARP	2.3.2						Y	
Flush out-of-date ARP cache entries	2.3.2.1	x					Y	
Prevent ARP floods	2.3.2.1	x					Y	
Cache timeout configurable	2.3.2.1		x				Y	
Save at least one (latest) unresolved pkt	2.3.2.2		x				Y	
Ethernet and IEEE 802 Encapsulation	2.3.3							
Host able to:	2.3.3							
Send and receive RFC-894 encapsulation	2.3.3	x					Y	
Receive RFC-1042 encapsulation	2.3.3		x				N	Only 802.3/Eth frames.
Send RFC-1042 encapsulation	2.3.3			x			N	
Then config. Sw. to select RFC-894 dftl	2.3.3	x					N	
Send K1=6 encapsulation	2.3.3					x	N	
Use ARP on Ethernet and IEEE 802 nets	2.3.3	x					Y	
Link layer report b'casts to IP-layer	2.4	x					Y	
IP layer pass TOS to link layer	2.4	x					Y	
No ARP entry treated as Destiation Unreachable.	2.4					x	N	
Implement IP and ICMP	3.1	x					Y	
Handle remote multihoming in application layer	3.1	x					Y	
Support local multihoming	3.1			x			Y	
Meet gateway specs if forward datagram	3.1	x					N	RTNET is a hos TCP/IP stack.
Configuration switch for embedded gateway	3.1	x					-	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Config switch default to non-gataway	3.1	x					-	
Auto-config based on number of interfaces	3.1					x	-	
Able to log discarded datagram	3.1		x				N	
Record in counter	3.1		x				N	
Silenty discard Version != 4	3.2.1.1	x					Y	
Verify IP checksum silently discard bad dgram	3.2.1.2	x					Y	
Addressing:								
Subnet addressing (RFC-950)	3.2.1.3	x					Y	
Src address must be host's own IP address	3.2.1.3	x					Y	
Silently discard dgram with bad destination address	3.2.1.3	x					Y	
Silently discard dgram with bad src addr	3.2.1.3	x					Y	
Support reassembly	3.2.1.4	x					Y	
Retain same Id field in identical datagram	3.2.1.5			x			N	
TOS:								
Allow transport layer to set TOS	3.2.1.6	x					N	
Pass received TOS up to transport layer	3.2.1.6		x				N	
Use RFC-795 link-layer mappings for TOS	3.2.1.6				x		N	
TTL:								
Send packet with TTL of 0	3.2.1.7					x	N	
Discard received packets with TTL <2	3.2.1.7					x	N	
Allow transport layer to set TTL	3.2.1.7	x					Y	
Fixed TTL is configurable	3.2.1.7	x					Y	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
IP Options:							N	IP-options is no supported.
Allow transport layer to send IP options	3.2.1.8	x					-	
Pass all IP options rcvd to higher layer	3.2.1.8	x					-	
IP layer silently ignore unknown options	3.2.1.8	x					-	
Security option	3.2.1.8a			x			-	
Send Stream Identifier option	3.2.1.8b				x		-	
Silently ignore Stream Identifier option	3.2.1.8b	x					-	
Record Route option	3.2.1.8d			x			-	
Timestamp option	3.2.1.8e			x			-	
Source Route Option:							-	
Originate and terminate Source Route options	3.2.1.8c	x					-	
Datagram with completed SR passed up to TL	3.2.1.8c	x					-	
Build correct (non-redudant) return route	3.2.1.8c	x					-	
Send multiple SR options in one header	3.2.1.8c					x	-	
ICMP:								
Silently discard ICMP message with unknown type	3.2.2	x					Y	
Include more than 8 octets of original datagram	3.2.2			x			N	
Include octets same as received	3.2.2	x					Y	
Demux ICMP Error to transport protocol	3.2.2	x					Y	
Send ICMP error message with TOS=0	3.2.2		x				Y	
Send ICMP error message for:								
- ICMP error message	3.2.2					x	N	
- IP b'cast or IP m'cast	3.2.2					x	N	

WNDRVR

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
- Link-layer b'cast	3.2.2					x	N	
- Non-initial fragment	3.2.2					x	N	
- Datagram with non-unique src address	3.2.2					x	N	
Return ICMP error messages (when not prohibited)	3.3.8	x					Y	
Dest Unreachable:								
Generate Dest Unreachable (code 2/3)	3.2.2.1		x				Y	
Pass ICMP Dest Unreachable to higher layer	3.2.2.1	x					Y	
Higher layer act on Dest Unreach	3.2.2.1		x				Y	
Interpret Dest Unreach as only hint	3.2.2.1	x					Y	
Redirect:							N	RTNET is a hos TCP/IP stack.
Host send Redirect	3.2.2.2			x			-	
Update route cache when rcv Redirect	3.2.2.2	x					-	
Handle both Host and Net Redirects	3.2.2.2	x					-	
Discard illegal Redirect	3.2.2.2		x				-	
Source Quench:							N	
Send Source Quench if buffering exceeded	3.2.2.3			x			-	
Pass Source Quench to higher layer	3.2.2.3	x					-	
Higher layer act on Source Quench	3.2.2.3		x				-	
Time Exceeded: pass to higher layer	3.2.2.4	x					N	
Parameter problem:								
Send Parameter Problem message	3.2.2.5		x				N	
Pass Parameter Problem to higher layer	3.2.2.5	x					N	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Report Parameter Problem to user	3.2.2.5			x			N	
ICMP Echo Request or Reply:								
Echo server and Echo client	3.2.2.6	x					Y	
Echo client	3.2.2.6		x				Y	
Discard Echo Request to broadcast address	3.2.2.6			x			N	
Discard Echo Request to multi-cast address	3.2.2.6			x			N	
Use specific-dest addr as Echo Reply src	3.2.2.6	x					Y	
Send same data in Echo Reply	3.2.2.6	x					Y	
Pass Echo Reply to higher layer	3.2.2.6	x					Y	
Reflect Record Route Time Stamp options	3.2.2.6		x				N	
Reverse and reflect Source Route option	3.2.2.6	x					N	
ICMP Information Request or Reply:	3.2.2.7				x		N	
ICMP Timestamp Request or Reply:	3.2.2.8			x			N	
Minimize delay variability	3.2.2.8		x				-	
Silent discard b'cast Timestamp	3.2.2.8			x			-	
Silent discard m'cast Timestamp	3.2.2.8	x		x			-	
Reflect Record Route Time Stamp options	3.2.2.6		x				-	
Reverse and reflect Source Route option	3.2.2.8	x					-	
Pass Timestamp Reply to higher layer	3.2.2.8	x					-	
Obey rules for "standard value"	3.2.2.8	x					-	
ICMP Address Mask Request and Reply:							N	
Addr Mask source configurable	3.2.2.9	x					N	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Support static configuration of addr mask	3.2.2.9	x					N	
Get addr mask dynamically during booting	3.2.2.9			x			N	
Get addr via ICMP Addr Mask Request/Reply	3.2.2.9			x			N	
Retransmit Addr Mask Req if no Reply	3.2.2.9	x					N	
Assume default mask if no Reply	3.2.2.9		x				N	
Update address mask from first Reply only	3.2.2.9	x					N	
Reasonableness check on Addr Mask	3.2.2.9		x				N	
Send unauthorized Addr Mask Reply mesg	3.2.2.9					x	N	
Explicitly configured to be agent	3.2.2.9	x					N	
Static config => Addr-Mask-Authoritative flag	3.2.2.9		x				N	
Broadcast Addr Mask Reply when init.	3.2.2.9	x					N	
Routing outbound datagrams:							N	RTNET is a hos TCP/IP stack.
Use address mask in local/remote decision	3.3.1.1	x					-	
Operate with no gateways on conn network	3.3.1.1	x					-	
Maintain "route cache" of next-hop gateways	3.3.1.2	x					-	
Treat Host and Net Redirect the same	3.3.1.2		x				-	
If no cache entry use default gateway	3.3.1.2	x					-	
Support multiple default gateway	3.3.1.2	x					-	
Provide table of static routes	3.3.1.2			x			-	
Flag: route over-ridable by Redirects	3.3.1.2			x			-	
Key route cache on host not net address	3.3.1.3			x			-	
Include TOS in route cache	3.3.1.3		x				-	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Able to detect failure of next-hop gateway	3.3.1.4	x					Y	
Assume route is good forever	3.3.1.4				x		N	
Ping gateway continuously	3.3.1.4					x	N	
Ping only when traffic being sent	3.3.1.4	x					N	The neighbor c state machine for reachability conformation.
Ping only when no positive indication	3.3.1.4	x					N	The neighbor c state machine for reachability conformation.
Higher and lower layers give advise	3.3.1.4		x					
Switch from failed default gateway to another	3.3.1.5	x					N	
Manual method of entering config info	3.3.1.6	x					Y	
Reassembly and Fragmentation								
Able to reassemble incoming datagram	3.3.2	x					Y	
At least 576 byte datagrams	3.3.2	x					Y	
EMTU_R configurable or indefinite	3.3.2		x				Y	
Transport layer able to learn MMS_R	3.3.2	x					Y	
Send ICMP Time Exceeded on reassembly timeout	3.3.2	x					N	
Fixed reassembly timeout value	3.3.2		x				Y	
Pass MSS_S_ to higher layers	3.3.3	x					Y	
Local fragmentation of outgoing packets	3.3.3		x				Y	
Else don't send bugger than MSS_S	3.3.3	x					Y	
Send max 576 to iff-net destination	3.3.3		x				Y	
All-Subnets-MTU configuration flag	3.3.3			x			N	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Multihoming								
Reply with same addr as spec-dest addr	3.3.4.2		x				Y	
Allow application to choose local IP addr	3.3.4.2	x					Y	
Silently discard d'gram in "wrong" interface	3.3.4.2			x			Y	
Only send d'gram through "right" interface	3.3.4.2			x			Y	
Source-Route Forwarding							N	
Forward datagram with Source Route option	3.3.5			x			-	
Obey corresponding gateway rules	3.3.5	x					-	
Update TTL gateway rules	3.3.5	x					-	
Able to generate ICMP err code 4, 5	3.3.5	x					-	
IP src addr not local host	3.3.5			x			-	
Update Timestamp Record Route options	3.3.5	x					-	
Configurable switch for non-lock SRing	3.3.5	x					-	
Defaults to OFF	3.3.5	x					-	
Satisfy gwy access rules for non-lock SRing	3.3.5	x					-	
If not forward send Dest Unreach (code 5)	3.3.5		x				-	
Broadcast							Y	
Broadcast addr as IP source addr	3.2.1.3					x	-	
Receive 0 or -1 broadcast formats OK	3.3.6		x				-	
Configurable option to send 0 or -1 b'cast	3.3.6			x			-	
Default to -1 broadcast	3.3.6		x				-	
Recognize all broadcast address formats	3.3.6	x					-	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Use IP b'cast/m'cast addr in link-layer b'cast	3.3.6	x					-	
Silently discard link-layer-only b'cast datagram	3.3.6		x				-	
Use Limit ed Broadcast addr for connected net	3.3.6		x				-	
Multicast							Y	
Support local IP multicasting (RFC-1112)	3.3.7		x				-	
Support IGMP (RFC-1112)	3.3.7			x			-	
Join all-hosts group at startup	3.3.7		x				-	
Higher layers learn I'face m'cast capability	3.3.7		x				-	
Interface								
Allow transport layer to use all IP mechanisms	3.4	x					Y	
Pass interface identification up to transport layer	3.4	x					Y	
Pass all IP options up to transport layer	3.4	x					N	IP-options not supported.
Transport layer can send certain ICMP messages	3.4	x					Y	
Pass spec'd ICMP messages up to transport layer	3.4	x					Y	
Include IP hdr+8 octets or more from orig.	3.4	x					Y	
UDP send port Unreachable	4.1.3.1		x				Y	
IP Options in UDP							N	IP-options not supported.
- Pass rcv-d IP options to applic layer	4.1.3.2	x					-	
- Applic layer can specify IP options in Send	4.1.3.2	x					-	
- UDP passes IP options down to IP layer	4.1.3.2	x					-	
Pass ICMP messages up to application layer	4.1.3.3	x					Y	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
UDP checksums:								
- Able to generate/check checksum	4.1.3.4	x					Y	
- Silently discard bad checksum	4.1.3.4	x					Y	
- Sender Option not to generate checksum	4.1.3.4			x			N	
- Default is to checksum	4.1.3.4	x					Y	
- Receiver Option to require checksum	4.1.3.4			x			N	
UDP Multihoming								
- Pass spec-dest addr to application	4.1.3.5	x					Y	
- Applic layer can specify Local IP addr	4.1.3.5	x					Y	
- Applic layer specify wild Local IP addr	4.1.3.5	x					Y	
- Applic layer notified of Local IP addr used	4.1.3.5		x				Y	
Bad IP src addr silently discarded by UDP/IP	4.1.3.6	x					Y	
Only send valid IP source addr	4.1.3.6	x					Y	
UDP Application Interface Service								
Full IP interface of 3.4 for application	4.1.4	x					N	
- Able to spec TTP, TOS, IP opts when send dg	4.1.4	x					N	
- Pass received TOS up to applic layer	4.1.4			x			N	
TCP								
Push flag								
Aggregate or queue un-pushed data	4.2.2.2			x			Y	
Sender collapse successive PSH flags	4.2.2.2		x				Y	
SEND call can specify PUSH	4.2.2.2			x			N	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
If cannot: sender buffer indefinitely	4.2.2.2					x	N	
If cannot: PSH last segment	4.2.2.2	x					Y	
Notify receiving APL of PSH	4.2.2.2			x			N	
Send max size segment when possible	4.2.2.2		x				Y	
Window								
Treat as unsigned number	4.2.2.3	x					Y	
Handle as 32-bit number	4.2.2.3		x				Y	
Shrink window from right	4.2.2.16				x		N	
Robust against shrinking window	4.2.2.16	x					Y	
Receiver's window closed indefinitely	4.2.2.17			x			Y	
Sender probe zero window	4.2.2.17	x					N	
First probe after RTO	4.2.2.17		x				-	
Exponential backoff	4.2.2.17		x				-	
Allow window stay zero indefinitely	4.2.2.17	x					Y	
Sender Timeout OK conn with zero wind	4.2.2.17					x	N	
Urgent Data								
Pointer points to last octet	4.2.2.4	x					Y	
Arbitrary length urgent data sequence	4.2.2.4	x					Y	
Inform APL asynchronously of urgent data	4.2.2.4	x					N	
APL can learn of/how much urgent data Q'd	4.2.2.4	x					N	
TCP Options								
Receive TCP option in any segment	4.2.2.5	x					Y	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Ignore unsupported option	4.2.2.5	x					Y	
Cope with illegal option length	4.2.2.5	x					Y	
Implement sending and receiving MSS option	4.2.2.6	x					Y	
Send MSS option unless 536	4.2.2.6		x				Y	
Send MSS option always	4.2.2.6			x			Y	
Send MSS default is 536	4.2.2.6	x					Y	
Calculate effective send seg size	4.2.2.6	x					Y	
TCP checksum								
Sender computes checksum	4.2.2.7	x					Y	
Receiver check checksum	4.2.2.7	x					Y	
Use clock-driven ISN selection	4.2.2.9	x					Y	Clock-driven a calculated on a flow basis for increased secu
Opening Connections								
Support simultaneous open attempts	4.2.2.10	x					Y	
SYN-RCVD remember last state	4.2.2.11	x					Y	
Passive Open call interfere with others	4.2.2.18					x	N	
Function: simultaneous LISTENs for same port	4.2.2.18	x					Y	
Ask IP for src addr for SYN if neccessary.	4.2.3.7	x					Y	
Otherwise use local addr of conn.	4.2.3.7	x					Y	
OPEN to broadcast/multicast IP Address	4.2.3.10					x	N	
Silently discard seg to bcast/mcast addr	4.2.3.10	x					Y	
Closing Connections								

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
RST can contain data	4.2.2.13		x				Y	
Inform application of aborted conn	4.2.2.13	x					Y	
Half-duplex close connections	4.2.2.13			x			Y	
Send RST to indicate data lost	4.2.2.13		x				N	
In TIME-WAIT state for 2xMSL seconds	4.2.2.13	x					Y	
Accept SYN from TIME-WAIT state	4.2.2.13			x			N	
Retransmissions								
Jacobson Slow Start Algorithm	4.2.2.15	x					Y	
Jacobson Congestion-Avoidance algorithm	4.2.2.15	x					Y	
Retransmit with same IP identification	4.2.2.15			x			N	
Karn's algorithm	4.2.3.1	x					Y	
Jacobson's RTO estimation alg.	4.2.3.1	x					Y	
Exponential backoff	4.2.3.1	x					Y	
SYN RTO calc same as data	4.2.3.1		x				Y	
Recommended initial values and bounds	4.2.3.1		x				Y	
Generating ACK's:								
Queue out-of-order segments	4.2.2.20		x				Y	
Process all Q'd before send ACK	4.2.2.20	x					N	
Send ACK for out-of-order segment	4.2.2.21			x			Y	
Delayed ACK's	4.2.3.2		x				Y	
Delay <0.5 seconds	4.2.3.2	x					Y	
Every 2nd full-sized segmentet ACK'd	4.2.3.2	x					Y	
Receiver SWS-Avoidance Algorithm	4.2.3.2	x					Y	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Sending data:								
Configurable TTL	4.2.2.19	x					Y	
Sender SWS-Avoidance algorithm	4.2.3.4	x					Y	
Nagle algorithm	4.2.3.4		x				Y	
Application can disable Nagle algorithm	4.2.3.4	x					Y	
Connection Failures:								
Negative advice to IP on R1 retxs	4.3.2.5	x					N	
Close connection on R2 retxs	4.3.2.5	x					Y	
APL can set R2	4.3.2.5	x					N	
Inform APL of R1 >= retxs < R2	4.3.2.5		x				N	
Recommended values for R1, R2	4.3.2.5		x				Y	
Same mechanism for SYNs	4.3.2.5	x					Y	
R2 at least 3 minutes for SYN	4.3.2.5	x					Y	
Send Keep-alive Packets:				x			N	
- Application can request	4.2.3.6	x					-	
- Default is OFF	4.2.3.6	x					-	
- Only send if idle for interval	4.2.3.6	x					-	
- Interval configurable	4.2.3.6	x					-	
- Default at least 2 hrs.	4.2.3.6	x					-	
- Tolerant of lost ACK's	4.2.3.6	x					-	
IP options:							N	IP-options not implemented.
Ignore options TCP doesn't understand	4.2.3.8	x					-	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Time Stamp support	4.2.3.8			x			-	
Record Route support	4.2.3.8			x			-	
Source Route:								
APL can specify	4.2.3.8	x					-	
Overrides src rt in datagram	4.2.3.8	x					-	
Build return route from src rt	4.2.3.8	x					-	
Later src route overrides	4.2.3.8		x				-	
Receiving ICMP message from IP	4.2.3.9	x					N	
Dest. Unreach (0, 1, 5) => inform APL	4.2.3.9		x				-	
Dest. Unreach (0, 1, 5) => abort conn	4.2.3.9					x	-	
Dest. Unreach (2-4) => abort conn	4.2.3.9		x				-	
Source Quench => slow start	4.2.3.9		x				-	
Time Exceeded => tell APL don't abort	4.2.3.9		x				-	
Param Problem => tell APL don't abort	4.2.3.9		x				-	
Address Validation:								
Reject OPEN call to invalid IP address	4.2.3.10	x					Y	
Reject SYN from invalid IP address	4.2.3.10	x					Y	
Silently discard SYN to bcast/mcast addr	4.2.3.10	x					Y	
TCP/APL Interface Service:								
Error Report mechanism	4.2.4.1	x					Y	
APL can disable Error Report Routine	4.2.4.1		x				N	
APL can specify TOS for sending	4.2.4.2	x					N	

Feature	Section	Must	Should	May	Should Not	Must Not	Compliance	Notes
Passed unchanged to IP	4.2.4.2		x				Y	
APL can change TOS during connection	4.2.4.2		x				N	
Pass received TOS up to APL	4.2.4.2			x			N	
FLUSH call	4.2.4.3			x			N	
Optional Local IP addr param. In OPEN	4.2.4.4	x					Y	

Parent topic: Supported RFCs on page 4

6. Supported BSPs and Drivers

The real-time network (RTNET) stack includes user mode driver support for specific network devices. The stack has also been verified for use with certain BSPs.

Supported User Mode Network Drivers

The following drivers are available for use in user mode (in an RTP) with the RTNET stack:

- GEI driver for Intel PRO/1000 PCIe devices
- ENET driver for Freescale I.MX6 ENET on-board devices

For more information on RTP-based (user mode) drivers, see the VxWorks 7 BSP and Driver Guide.

BSP Support

The RTNET stack is not limited to a specific set of VxWorks BSPs with the exception that it cannot be used with VxWorks 6.9-compatible BSPs. The stack supports multiple CPU architectures, addressing modes (32- and 64-bit) and multiprocessor configurations (uniprocessor (UP) and symmetric multiprocessor (SMP)). The following BSPs have been verified to run with the RTNET stack:

- Freescale i.MX6
- Freescale P1/P2
- Intel Generic
- VxSim for Linux

For a complete list of supported BSPs, see your product release notes.

Related information

Real-Time Network Stack Overview on page 1
User Mode I/O System on page 2
Supported Protocols on page 3
Supported Socket Types on page 3
Supported RFCs on page 4

WNDRVR

Real-Time Network Stack Limitations on page 22

7. Real-Time Network Stack Limitations

The real-time network stack (RTNET) has not been verified with any of the existing network applications in VxWorks 7, including the debugging tools.

Internet Control Message Protocol (ICMP) implementation is the bare minimum for operation on IP networks and to handle ECHO-request/response.

Only one AF_INET/SOCK_RAW socket can be created per Internet protocol. The only practical implication is that just one instance of the ping4 command can be active at any point in time. You can have multiple external ICMP-echo request/response sessions active, as the restriction only applies when running ping4 on the target in the VxWorks shell.

Related information

Real-Time Network Stack Overview on page 1
User Mode I/O System on page 2
Supported Protocols on page 3
Supported Socket Types on page 3
Supported RFCs on page 4
Supported BSPs and Drivers on page 21

2. VXWORKS AND RTNET CONFIGURATION

1. Configuring VxWorks and RTNET

VxWorks can be configured for using RTNET in the kernel, as well as in user-space RTP applications. A single instance of RTNET can be used in the kernel with multiple RTP instances, simultaneously.

Before You Begin

Take the following guidelines into consideration when you design your system, and when you configure VxWorks for RTNET:

- The RTNET stack and the IPNET stack are mutually exclusive in the kernel (**INCLUDE_RTNET** and **INCLUDE_IPNET**). You can, however, use RTNET in an RTP application with either RTNET or IPNET in the kernel, in the same system.
- The RTNET stack cannot be used with **INCLUDE_DEBUG_AGENT** or **INCLUDE_STOP_MODE_AGENT** (which are included with the **PROFILE_DEVELOPMENT** configuration profile). These components require IPNET.
- The RTNET RTP auto-start facility cannot be used in a system that is configured with either RTNET or IPNET in the kernel. The **INCLUDE_RTNET_RTP_AUTOSTART** component is incompatible with both **INCLUDE_RTNET** and **INCLUDE_IPNET**.

Procedure

1. Create a VSB project and add RTNET support.
   - For kernel-space RTNET support, add RTNET.
   - For RTP RTNET support, add RTNET_RTP.
2. Add the driver layer required for your target hardware.

   For example:

   - For the user space version of the Intel PRO/1000 network (END) driver, add RTNET_END_DRIVER_GEI.
   - For the user space version of the Freescale I.MX ENET network (END) driver, add RTNET_END_DRIVER_ENET.

   For VxSim, no driver layer is required.

3. Build the VSB.

   The VSB build output includes demo RTP applications.

4. Create a VIP project based on your VSB.

   CAUTION: **Do not base your project on the PROFILE_DEVELOPMENT profile.**

5. For IPv4 support, add **INCLUDE_RTNET_IPV4_SUPPORT** (which is the default). For IPv6 support, add **INCLUDE_RTNET_IPV6_SUPPORT**.
6. For RTNET in the kernel, optionally set the **RTNET_JSON_FILE** parameter to a JSON configuration file.

   If the configuration parameter is not set to a valid JSON file, the default RTNET settings are used. See RTNET JSON Configuration File Reference on page 27.

7. To use RTNET in an RTP application:
   1. If you want to use networking in the kernel, in addition to your RTP, add **INCLUDE_RTNET** or **INCLUDE_IPNET**.

WNDRVR

2. If you are not going to use the RTP auto-start method, add **INCLUDE_ENV_VARS**. This will allow you to set the RTNET environment variable to the location of the RTNET JSON configuration file (for the stack's boot device, IP address, and so on) when you launch the RTP application.
3. If you are using the VxWorks simulator (VxSim), and are not going to use the RTP auto-start method, you must set the **IFCONFIG_1** configuration parameter to an empty string. This means that you must add **INCLUDE_IPNET**, which provides IPNET in the kernel. It also means that you cannot use RTNET in the kernel.
4. If you are going to use the RTP auto-start method, you cannot use RTNET or IPNET in the kernel, so you must remove **INCLUDE_RTNET** or **INCLUDE_IPNET** if either of them has been included in your project.
5. Finally for the RTP auto-start method, add the **INCLUDE_RTNET_RTP_AUTOSTART** component and configure the the **RTNET_RTP_EXEC_PATH** and **RTNET_RTP_EXEC_PARAMS** parameters.
6. Add storage support for your RTP applications (for example, ROMFS).

8. Add the driver component required for your target.

   For example, for the i.MX6 Sabrelite, add **INCLUDE_FSL_IMX6Q_SABRELITE**.

   Note: You must add **INCLUDE_RTNET_MUX** in either of these cases:
   - If your target is the VxWorks simulator (VxSim).
   - If you want to use RTNET in an RTP application and there is no user-mode (RTP) network driver available for the target.

9. Optionally, add **INCLUDE_RTP_SHELL_CMD** and **INCLUDE_DISK_UTIL_CMD**.

   The kernel shell command interpreter is convenient for launching RTP applications, and disk utilities for file system navigation, and so on. They are required if you want to use the examples described later in this document.

10. Optionally, add additional RTNET components.

    For information about components for the kernel show routines, ping, demo programs, the MUX, and vSwitch, see VxWorks RTNET Configuration Reference on page 24

11. Build your VIP.

Related information

VxWorks RTNET Configuration Reference on page 24
RTNET JSON Configuration File Reference on page 27

2. VxWorks RTNET Configuration Reference

To enable support for the real-time network (RTNET) stack, you must set the required configuration options and parameters in your VxWorks source build (VSB) projects and VxWorks image projects (VIPs).

Note: You cannot configure VxWorks with both the RTNET stack and the standard VxWorks network stack (IPNET).

For information on building and configuring VxWorks, see the *VxWorks 7 Configuration and Build Guide*.

VSB Configuration Options

VSB Configuration Option	Description
RTNET	Provides support for the RTNET stack in your VxWorks source build.
RTNET_RTP	Provides support for using the RTNET stack in a real-time process (RTP).
RTNET_END_DRIVER_GEI	Provides support for the user space version of the Intel PRO/1000 network (END) driver.
RTNET_END_DRIVER_ENET	Provides support for the user space version of the Freescale I.MX ENET network (END) driver.

VIP Configuration Components and Parameters

Component	Description
INCLUDE_RTNET	Provides support for the real-time network stack in the VxWorks kernel. (Excludes **INCLUDE_RTNET_RTP_AUTOSTART**.) Provides the following configuration parameter: **RTNET_JSON_FILE** Sets the pathname of the Javascript object notation (JSON) configuration file. The default assignment for the **RTNET_JSON_FILE** parameter of the **INCLUDE_RTNET** component is an empty string. For information about the configuration file, see RTNET JSON Configuration File Reference on page 27.
INCLUDE_RTNET_FIB_SHOW	Provides the fibShow command which prints the content of the forwarding information base (FIB). For use with the kernel RTNET stack.
INCLUDE_RTNET_LINK_SHOW	Provides the linkShow command which prints information about all attached network devices. For use with the kernel RTNET stack.
INCLUDE_RTNET_IP_SHOW	Provides the ip4Show and ip6Show commands which print information about IP addresses. For use with the kernel RTNET stack.
INCLUDE_RTNET_NEIGH_SHOW	Provides the neighShow command which prints active neighbors. For use with the kernel RTNET stack.
INCLUDE_RTNET_SOCK_SHOW	Provides the sockShow command which prints the open sockets. For use with the kernel RTNET stack.
INCLUDE_RTNET_PING	Provides the ping4 and ping6 commands. For use with the kernel RTNET stack.

Component	Description
INCLUDE_RTNET_DEMO	Provides kernel demo applications such as blaster/blastee and the echo server. Not required for RTP demo applications, which are automatically built as part of the VSB build.
INCLUDE_RTNET_RTP_AUTOSTART	Provides automatic startup of the RTNET stack RTP with network configuration taken from boot parameters. (See RTNET Stack Startup in an RTP on page 36.) Provides the following configuration parameters: **RTNET_RTP_EXEC_PATH** Sets the path to the RTNET stack RTP executable. The default value is **"/romfs/telnetd.vxe"**. **RTNET_RTP_EXEC_PARAMS** Sets the application parameters to pass to the RTP executable. The default value is "". **INCLUDE_RTNET_RTP_AUTOSTART** and **INCLUDE_RTNET** are mutually exclusive.
INCLUDE_RTNET_DEBUG	Provides a printout of debug information related to the network stack. Only functions when **INCLUDE_RTNET_RTP_AUTOSTART** is included.
INCLUDE_RTNET_MUX	Provides an adapter for using the RTNET stack in an RTP that allows the stack to use of MUX devices in the kernel. This component allows for testing and evaluation if a user-mode driver has not been developed yet. This component is required for VxWorks simulator targets (VxSim).
INCLUDE_RTNET_VSWITCH	Provides a virtual Ethernet switch between stack instances in RTPs. Provides the following configuration parameter: **RTNET_RTP_EXEC_PARAMS** Sets the maximum number of instances that can be attached to the vswitch. The default value is 2.
INCLUDE_RTNET_SAFETY_CERT_RTP	Provides configuration of the real-time network stack for the Safety Profile, and starting the stack automatically. Provides the following configuration parameters: **RTNET_JSON_FILE** Sets the path to the real-time network stack JSON configuration file The default value is **"/romfs/rtnet.json"**. For information about the configuration file, see RTNET JSON Configuration File Reference on page 27. **RTNET_RTP_EXEC_PATH** Sets the path to the RTNET stack RTP executable. The default value is **"/romfs/telnetd.vxe"**.

Related information

Configuring VxWorks and RTNET on page 23
RTNET JSON Configuration File Reference on page 27

3. RTNET JSON Configuration File Reference

The real-time network (RTNET) stack is configured using a Javascript object notation (JSON) file. You can change the default stack settings by defining key/value pairs in this file.

JSON File

When the RTNET stacks starts, it retrieves configuration parameters from a JSON configuration file. All JSON objects are optional.

> Note: Any IPv4 or IPv6 configuration set in the JSON file overrides the VIP configuration provided by **INCLUDE_RTNET_IPV4_SUPPORT** or **INCLUDE_RTNET_IPV6_SUPPORT**.

If a particular configuration parameter is not found in the JSON file (or if a JSON file is not identified and found), VxWorks uses the default compile time settings for configuration. The defaults are described in this reference.

The exceptions are as follows:

- You cannot use a JSON file with **INCLUDE_RTNET_RTP_AUTOSTART**. If you have configured VxWorks with **INCLUDE_RTNET_RTP_AUTOSTART**, the RTNET stack is started automatically using the default RTNET settings and the network settings from the VxWorks boot parameters.
- For **INCLUDE_RTNET_SAFETY_CERT_RTP** you must include network settings in the JSON file because the Safety Profile does not use boot parameters.

The location of the JSON file is identified with the **RTNET_JSON_FILE** configuration parameter (for both **INCLUDE_RTNET** and **INCLUDE_RTNET_SAFETY_CERT_RTP**).

Example JSON files are provided in *installDir* **/vxworks-7/pkgs_v2/net/rtnet/demo/jsonconfig**.

JSON Object Format

The format of a JSON object is as follows:

```
"object1" : {
    "name1" : value ,
    "name2" : value ,
    "name3" : value
}
```

JSON data is written as a set of key/value pairs. A key/value pair consists of a field name (in double quotes), followed by a colon, followed by a value. The JSON value can be a decimal number, a string, or a Boolean. You only need to specify the key/value pairs that you want to change from the default. For example, the JSON object "*object1*" can be specified as follows to update only the "*name2*" data:

```
"object1" : {
    "name2" : 1
}
```


WNDRVR

JSON Configuration File Example

The following example is of a complete JSON configuration file that sets all recognized objects to the default values and adds one interface to the network stack. Curly braces hold objects and square brackets hold arrays. The file must begin and end with curly braces. Note that comments are not supported in the JSON format.

```
{
    "task": {
        "priority": 50,
        "options" : 0,
        "stacksz" : 8192
    },
    "fib": {
        "capacity": 50
    },
    "peerinfo": {
        "capacity": 64,
        "lifetime": 60
    },
    "reassembly": {
        "capacity": 4,
        "lifetime": 15
    },
    "protocol": {
        "portlo": 49152,
        "porthi": 65535,
        "inet"  : true,
        "tcp"   : true,
        "udp"   : true
    },
    "netifs": [
        {
            "device": {
                "name": "enet",
                "unit": 0,
                "id": "ethernet0",
                "mac": "00:04:9f:b0:b1:b2"
            },
            "inet": {
                "address"  : "192.168.0.1",
                "prefixlen": 24,
                "gateway"  : "192.168.0.254"
            }
        }
    ]
}
```

The RTNET stack JSON objects and arrays are defined below.

task Object

The task object describes settings for the network stack task used for ingress packet processing. Typically, you do not need to update the task object values.

```
    "task": {
        "priority": <priority 1-255 for the network task> ,
        "options" : <VxWorks task options as integer> ,
        "stacksz" : <network task stack size in bytes>
    }
```

The default settings are as follows:

```
    "task": {
        "priority": 50,
        "options" : 0,
        "stacksz" : 8192
    }
```

fib Object

The fib object describes settings for the network stack forwarding information base.

```
    "fib": {
        "capacity": <maximum number of FIB entries as integer>
    }
```

The default settings are as follows:

```
    "fib": {
        "capacity": 50
    }
```

peerinfo Object

The peerinfo object describes settings for the network stack peer information base.

```
    "peerinfo": {
        "capacity": <maximum number of peers as integer> ,
        "lifetime": <lifetime in seconds for peer entries>
    }
```

The default settings are as follows:

```
    "peerinfo": {
        "capacity": 64,
        "lifetime": 60
    }
```

reassembly Object

The reassembly object describes settings for the network stack reassembly database.

```
    "reassembly": {
        "capacity": <maximum number of reassembly chains as integer> ,
        "lifetime": <lifetime in seconds for reassembly chains>
```

```
    }
```

The default settings are as follows:

```
    "reassembly": {
        "capacity": 4,
        "lifetime": 15
    }
```

protocol Object

The protocol object describes settings for the network stack protocol support.

```
    "protocol": {
        "portlo": <lowest ephemeral port, integer between 1-65535> ,
        "porthi": <highest ephemeral port, integer between 1-65535> ,
        "inet"  : <true|false, whether IPv4 is enabled or not> ,
        "inet6"  : <true|false, whether IPv6 is enabled or not> ,
        "tcp"   : <true|false, whether TCP is enabled or not> ,
        "udp"   : <true|false, whether UDP is enabled or not>

    }
```

The default settings are as follows:

```
    "protocol": {
        "portlo": 49152,
        "porthi": 65535,
        "inet"  : true,
        "tcp"   : true,
        "udp"   : true
    }
```

netifs Array

The netifs array describes network interfaces that will be attached to the network stack when it is started. Multiple network interfaces can be attached by specifying multiple objects in the array.

```
"netifs": [
          <array of JSON-objects as described below>
    ]

{
    "device": {
        "name": "<interface name (mandatory)> ",
        "unit": <interface unit number as integer (mandatory)> ,
        "mux" : "<mux name (mandatory when 'name' == 'mux')> ",
        "id"  : "<VxBus device identifier> "
        "mac" : "<Device MAC address> "
    },
        "inet": {
```

```
        "address"   :  "<dotted-quad>",
        "prefixlen": <prefix length as an integer 1-32>,
        "gateway"   :  "<dotted-quad>"
        }
}
```

The netifs array has no default settings. This means that for user-mode RTNET (in an RTP application), no interfaces are attached by default. For kernel mode, the boot interface that is specified with the boot parameters is attached if no interface is specified in the JSON configuration.

Each JSON object must be separated with a comma.

When RTNET is used in the kernel only, the device object has special requirements. The name and unit values are used. However, the mux, id, and mac values are no-ops. name and unit form the name of the interface in RTNET (for example, **gei0**). The name and unit values must also correspond to a device that is added to the MUX/END subsystem during the VxWorks kernel boot process. The name and unit are formed by the network (END) driver name and the unit number of the device.

When RTNET is used in an RTP, all key/value pairs in the device object apply and are defined as follows:

Key	Description
name	This key is mandatory. It describes the RTNET interface name when combined with unit. There are two special names. First, the name value of **mux** is used to indicate the driver for this interface is a MUX kernel driver. (You must include **INCLUDE_RTNET_MUX** to be able to use a MUX kernel driver, which is required for VxSim or to use RTNET in user mode when there is no user-mode network driver available for the target.) Second, the name value of **veth** indicates the interface is a virtual Ethernet interface using the virtual switch. If name is not one of the two special values, it can be any value less than 16 characters (when combined with unit) and sets the interface name in RTNET.
unit	This key is mandatory. This value describes the interface unit number. Combined with name it forms the interface name in RTNET; name+unit.
mux	Mandatory when the name value is **mux**. Otherwise, not used. Sets the MUX device name when combined with unit. Requires **INCLUDE_RTNET_MUX**.

id	Mandatory when name is not **mux**. Sets the VxBus device ID for the network device to be used for the interface. The VxBus device ID can be found using the devs shell command.
	The following are some examples:
	`"id": "ethernet0"`
	`"id": "pci device(8086:10d3)0"`
	`"id": "pci device(8086:105e)1"`

mac	Optional. Used to set the MAC address of the device. This value is only required for non-PCI Ethernet devices defined in the flattened device tree (FDT) file. For example: `"mac": "00:04:9f:b0:b1:b2"`

The following example adds three interfaces (one Intel PRO/1000 PCIe (GEI) 82574 and two Intel PRO/1000 PCIe (GEI) 82571):

```
"netifs": [
{
"device": {
  "name": "enet",
  "unit": 0,
  "id": "ethernet0",
  "mac": "00:04:9f:b0:b1:b2"
  },
"inet": {
  "address" : "192.168.0.1",
  "prefixlen": 24,
  "gateway" : "192.168.0.254"
  },
"inet6": {
  "address": "1::1",
  "prefixlen": 64,
  "gateway": "1::33"
  }
}
]
```

Related information

Configuring VxWorks and RTNET on page 23
VxWorks RTNET Configuration Reference on page 24

3. SOCKET APPLICATION FOR RTNET

1. BSD Socket Application Development

VxWorks provides the BSD 4.4 standard **sockLib** library for application development.

Socket applications can be created for various system configurations:

- as kernel applications (using DKM projects) that use the RTNET stack in the kernel
- as RTP applications (using RTP projects) that make system calls to the RTNET stack in the kernel
- as RTP projects that are linked with RTNET stack libraries so that both execute in an RTP (in user mode)

A socket application template is provided in *installDir* **/vxworks-7/pkgs_v2/net/rtnet/demo/sockapp**.

For detailed information on VxWorks sockets, see the *VxWorks 7 Sockets Programmer's Guide*.

C Header File for Socket Applications

For both kernel and RTP socket applications, the basic include statement required is for **sockLib.h**.

RTP Socket Application With User-Mode RTNET Stack

For RTP applications that use the RTNET stack in an RTP, they must:

- Link with **librtnet.a**, **libnet_base.a**, **libvxbus.a**, and **libjson.a**.
- Use the linker flag -u rtnetStackEnable.

If you are using Workbench, you can configure your project by editing the its build properties. See Configuring an RTP Project with RTNET in Workbench on page 34 for detailed instructions.

If you are working from the command line, use the following:

```
ADDED_LIBS += -lrtnet -lnet_base -lvxbus -ljson
```

```
LD_EXEC_FLAGS += -u rtnetStackEnable
```

Note: The RTNET stack is only available as a static library (**.a**) and not as a dynamically linked library (**.so**).

Optional VSB Build for RTP Socket Applications

If you would find it useful to build your RTP socket application as part of your VSB build, instead of as a standard RTP project, see the example makefile in in *installDir* **/vxworks-7/pkgs_v2/net/rtnet/demo/sockapp**. Install your makefile in your VSB project with your application code. Wind River uses this method to build various demo and test RTP applications by default in your VSB project (for example, **ping.vxe**).

Related information

Configuring an RTP Project with RTNET in Workbench on page 34

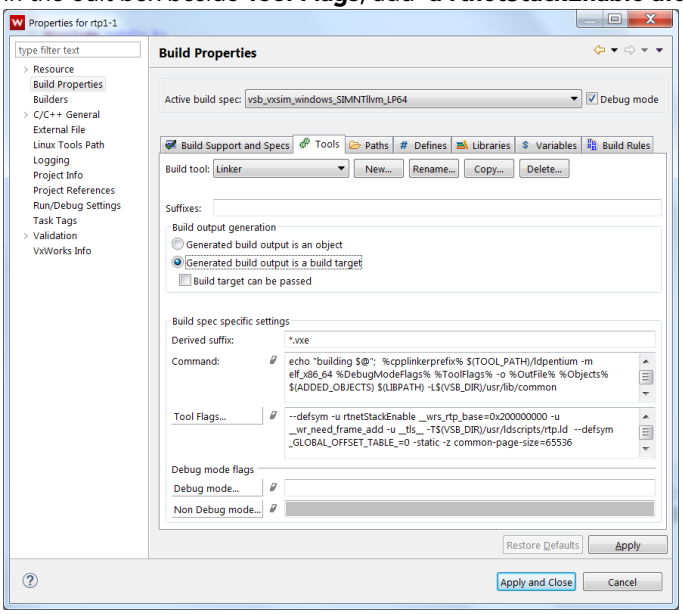
2. Configuring an RTP Project with RTNET in Workbench

About This Task

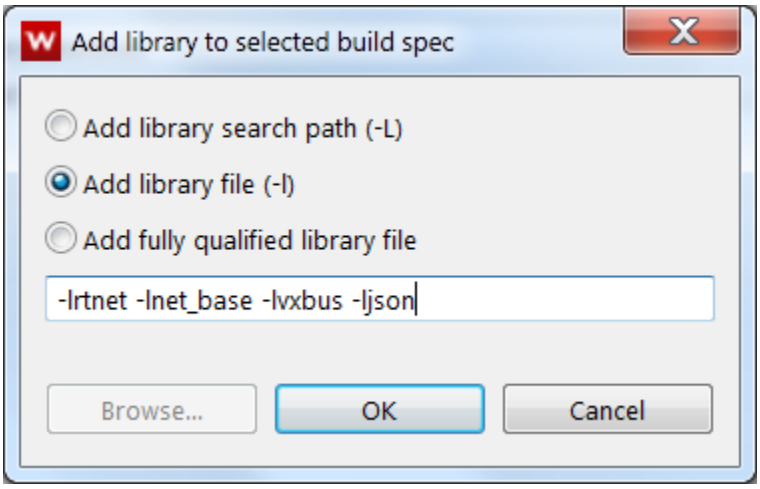
If you are configuring an RTP project with RTNET, you must set the -u rtnetStackEnable linker flag and add the required libraries. The following procedure provides detailed instructions.

Procedure

1. Select **File > Properties > Build Properties**.
2. Click the **Tools** tab.
3. For **Build tool**, select **Linker**.
4. In the edit box beside **Tool Flags**, add **-u rtnetStackEnable after --defsym**.



5. Click **Apply**.
6. Click the **Libraries** tab.
7. Click **Add**
8. In the Add library to selected build spec dialog, select **Add library file (-l)**.
9. In the edit field, type **-lrtnet -lnet_base -lvxbus -ljson** and click **OK**.



10. In the Build Properties dialog, click **Apply and Close**.

Related information

BSD Socket Application Development on page 33

4. STARTING THE RTNET STACK

1. RTNET Stack Startup in the Kernel

If you have configured the network stack to run in the kernel (with **INCLUDE_RTNET**) it will start automatically at boot-up.

The stack's configuration parameters (such as the network device, IP address, and so on), can be set with a JSON file or taken from the VxWorks boot parameters. The **RTNET_JSON_FILE** configuration parameter is used to identify a JSON file that contains the stack configuration parameters.

If **RTNET_JSON_FILE** is not set, then the VxWorks boot parameters are used for network settings. The RTNET stack defaults are used for the other settings. For more information, see RTNET JSON Configuration File Reference on page 27.

2. RTNET Stack Startup in an RTP

An RTNET RTP application can be started automatically at boot time, interactively from the shell, or programmatically.

Automatically Starting the RTNET Stack in an RTP

The **INCLUDE_RTNET_RTP_AUTOSTART** component can be used to start a single instance of the stack in an RTP. This method, however, does not allow use of a JSON configuration file. The RTNET stack is started automatically using the network settings from the VxWorks boot parameters and the stack defaults (which are set with configuration parameters) for the other settings. For more information, see RTNET JSON Configuration File Reference on page 27.

Alternatively, you could start one or more instances of the stack automatically at boot time using usrRtpAppInit( ).

Interactively Starting the RTNET Stack in an RTP

In order to start the RTNET stack interactively from the shell, as part of an RTP application:

- Your VxWorks configuration must include the **INCLUDE_ENV_VARS** component and **INCLUDE_RTP_SHELL_CMD**, in addition to basic RTNET RTP support.
- You must create and configure an JSON configuration file (see RTNET JSON Configuration File Reference on page 27) to set up network interfaces, and make it accessible to your target (in a target file system or on the host).
- At runtime, the RTNET environment variable must be set to point to the JSON configuration file.

For example, to start the RTP demo application **rtnet.vxe** using the interface information provided in a JSON configuration file stored in ROMFS, the kernel shell command would look like this:

```
-> putenv "RTNET_FILE=/romfs/gei.json"
-> rtpSp "/romfs/rtnet_app.vxe"
```

Or for the kernel command shell it would look like this:

```
-> cmd
[vxWorks *]# rtp exec -e RTNET_FILE=/romfs/gei.json /romfs/rtnet_app.vxe &
```

Multiple instances of the RTNET stack are started in different RTPs using different JSON configuration files to define the network interfaces of each. For example:

```
[vxWorks *]# rtp exec -e RTNET_FILE=/romfs/gei1.json /romfs/rtnet_app1.vxe &

[vxWorks *]# rtp exec -e RTNET_FILE=/romfs/gei2.json /romfs/rtnet_app2.vxe &
```

Note: These examples assume that the JSON configuration file and **.vxe** reside in ROMFS. The files can be stored in a different file system, such as RAM or flash, depending on your requirements.

Programmatically Starting the RTNET Stack in an RTP

Starting the RTNET stack in an RTP programmatically is similar to starting it from the kernel shell, but your code must set the RTNET environment variable with putenv( ) or setenv( ), and then use rtpSpawn( ) to execute the application.

5. KERNEL SHELL COMMANDS

1. Kernel Shell Commands

The real-time network (RTNET) stack provides support for a several kernel shell commands with VIP components. The kernel shell commands can only be used with an RTNET stack that has been implemented in the kernel (and not in an RTP).

fibShow

The fibShow command displays forward information base entries. For example:

```
-> fibShow
INET route table
192.168.0.6/24 dev gei0 src 192.168.0.6
127.0.0.1/8 dev lo src 127.0.0.1
default via 192.168.0.254 dev gei0 src 192.168.0.6

INET6 route table
::1/128 dev lo src ::1
1::1/64 dev simnet0 src 1::1
default via 1::33 dev simnet0 src 1::1
value = 6 = 0x6
```

This command is provided by the **INCLUDE_RTNET_FIB_SHOW** component.

ip4Show

The ip4Show command displays the IP addresses. For example:

```
-> ip4Show
lo: <UP,BROADCAST,RUNNING>  ifindex 1  mtu 1500  state UP
    inet 127.0.0.1/8  brd 127.255.255.255  <permanent>
gei0: <UP,BROADCAST,RUNNING>  ifindex 2  mtu 1500  state UP
    inet 192.168.0.6/24  brd 192.168.0.255  <permanent>
```

This command is provided by the **INCLUDE_RTNET_IP_SHOW** component.

linkShow

The linkShow command displays the interfaces table and interface statistics. For example:

```
-> linkShow
lo: <UP,BROADCAST,RUNNING>  ifindex 1  mtu 1500  state UP
        type loopback 00:00:00:00:00:00  brd ff:ff:ff:ff:ff:ff
                  bytes    packets     mcasts     errors
         RX:          0          0          0          0
         TX:          0          0          0          0
gei0: <UP,BROADCAST,RUNNING>  ifindex 2  mtu 1500  state UP
        type ethernet b8:ae:ed:7d:ab:30  brd ff:ff:ff:ff:ff:ff
                  bytes    packets     mcasts     errors
```


WNDRVR

```
        RX:       3784         39          0          0
        TX:       3766         39          0          0
```

This command is provided by the **INCLUDE_RTNET_LINK_SHOW** component.

neighShow

The neighShow command displays the neighbor table. For example:

```
-> neighShow
192.168.0.254   dev gei0            lladdr 00:1e:67:51:34:bf  REACHABLE
```

This command is provided by the **INCLUDE_RTNET_NEIGH_SHOW** component.

ping4 and ping6

The ping4 and ping6 commands check connectivity with other Internet hosts. For example:

```
-> ping4 "192.168.200.1"
```

```
-> ping4 "1::1"
```

Type **Ctrl-C** to stop the ping4 or ping6 command.

The ping commands are provided by the **INCLUDE_RTNET_PING** component.

sockShow

The sockShow command displays the active sockets.

```
-> sockShow
Proto  Rcv-Q  Snd-Q  Local Address           Remote Address          State
nlrt   0      0      18c528e4                -
tcp    0      0      192.168.0.6:23          192.168.0.254:33813     ESTABLISHED
tcp    0      0      0.0.0.0:5001            0.0.0.0:*               LISTEN
udp    0      0      0.0.0.0:5000            0.0.0.0:*
tcp    0      0      0.0.0.0:5000            0.0.0.0:*               LISTEN
udp    0      0      0.0.0.0:7               0.0.0.0:*
tcp    0      0      0.0.0.0:7               0.0.0.0:*               LISTEN
tcp    0      0      0.0.0.0:23              0.0.0.0:*               LISTEN
```

This command is provided by the **INCLUDE_RTNET_SOCK_SHOW** component.

6. DEMO APPLICATIONS AND EXAMPLE CODE

1. Kernel Demo Applications

The real-time network (RTNET) stack comes with a set of kernel demo applications for test and evaluation purposes.

The **INCLUDE_RTNET_DEMO** component provides the following demo applications that can be used with RTNET in the kernel:

- telnet server
- echo server
- blaster/blastee

These demo applications are started automatically after the RTNET stack is started in the VxWorks boot process.

The source code is in *installDir* **/vxworks-7/pkgs_v2/net/rtnet/demo/src**.

Telnet Server

The telnet server can be used to remotely access the kernel shell of a running VxWorks target. From a remote host, you can connect to the telnet server on port 23 using a telnet client which presents you with the shell prompt. For example:

```
$ telnet 192.168.0.6

Trying 192.168.0.6...
Connected to 192.168.0.6.
Escape character is '^]'.

-> version
VxWorks 7 (for x86 Processor (ACPI_BOOT_OP)).
Core Kernel version: 1.2.1.0.
Made on Jul  7 2016 14:18:28.
Boot line:
gei(0,0)host:vxWorks h=192.168.0.254 e=192.168.0.6:ffffff00 g=192.168.0.254 f=0x80
value = 94 = 0x5e = '^'

-> exit
Au revoir!
Connection closed by foreign host.
```

Echo Server

The echo server listens to port 7 on both the UDP and TCP protocol. The echo server returns everything it receives back to the sender. You can use a network utility tool on a remote host to send data to the echo server and the data is returned immediately.

For example, Linux hosts provide the netcat utility which can be used to send and receive data. The user enters the first string after the nc command has executed, and it is echoed by the echo server on the target.

```
$
Echo this TCP data!
Echo this TCP data!
^C
```

```
$ nc -u 192.168.0.6 7

Echo this UDP data!

Echo this UDP data!
^C
```

Blaster/Blastee

The blaster/blastee applications are used to examine network throughput performance. The blaster sends as much data it can while the blastee receives as much data it can. One convenient way to send data to the blastee and to receive data from the blaster is to use the netcat utility on a Linux host.

Netcat is a versatile TCP/IP application that is easy to configure and use. It can also source data from the **/dev/zero** device and sink data to the **/dev/null** device. When the blaster/blastee applications are running and data is received or transmitted, the current rate is displayed every ten seconds on the target shell.

For example, the blastee listens to TCP port 5000 and the blaster listens to TCP port 5001. In order to test receive throughput, you can run the following netcat command on the Linux host:

```
$ nc 192.168.0.6 5000 < /dev/zero
```

Observe the following output on the target shell:

```
-> blastTx: 0 bytes/sec (0 Mbit)
blastRx: 10060759 bytes/sec (123 Mbit)

blastTx: 0 bytes/sec (0 Mbit)
blastRx: 92852002 bytes/sec (986 Mbit)

blastTx: 0 bytes/sec (0 Mbit)
blastRx: 92676620 bytes/sec (986 Mbit)

blastTx: 0 bytes/sec (0 Mbit)
blastRx: 17295991 bytes/sec (986 Mbit)
```

In order to test the TCP send throughput of the RTNET stack, run the following netcat command on the Linux/BSD host:

```
$ nc 192.168.0.6 5001 > /dev/null
```

Observe the following output on the target shell:

```
blastTx: 51405654 bytes/sec (434 Mbit)
blastRx: 0 bytes/sec (0 Mbit)

blastTx: 116691621 bytes/sec (986 Mbit)
blastRx: 0 bytes/sec (0 Mbit)

blastTx: 116698174 bytes/sec (986 Mbit)
blastRx: 0 bytes/sec (0 Mbit)
```

You can abort the netcat application by pressing **Ctrl-C** in the Linux host shell.

Related information
RTP Demo Applications on page 42
Multicast Receive and Send Example on page 43

2. RTP Demo Applications

The real-time network (RTNET) stack comes with a set of RTP demo applications for test and evaluation purposes.

The following RTP demo applications are provided for use with RTNET in user mode:

- **echo.vxe**
- **blaster.vxe**
- **blastee.vxe**
- **netshell.vxe**
- **telnetd.vxe**

The applications are built with the VSB build, in *vsbDir* **/usr/root/***compiler* **/bin**.

The source code is in sub-directories of *installDir* **/vxworks-7/pkgs_v2/net/rtnet/demo/**.

The **echo.vxe**, **blaster.vxe**, **blastee.vxe**, and **telnetd.vxe** applications are similar to the kernel demo applications. They can be started in the standard manner for RTP applications.

For example:

```
[vxWorks *]# rtp exec -e RTNET_FILE=/romfs/gei.json /romfs/echo.vxe -- -s &

Launching process '/romfs/echo.vxe' ...
Process '/romfs/echo.vxe' (process Id = 0x801107b8) launched.
Attachment number for process '/romfs/echo.vxe' is %1.
```

This launch command passes the -s argument to the UDP echo server. See the source code for the available options.

The **netshell.vxe** application is more complex than the other demo applications, and is described in detail below.

RTP Network Shell Demo Application

When you run the RTNET stack in an RTP, the network shell commands implemented for the kernel shell (such as the show routines) are not available because they can only access a network stack instance in the kernel. To use the network shell commands inside an RTP, the functions must be called within the RTP. The **netshell.vxe** demo application provides this functionality.

Launch the application using the the -i option with the rtp exec command. For example:

```
[vxWorks *]# rtp exec -i -e RTNET_FILE=/romfs/gei.json /romfs/netshell.vxe

Launching process '/romfs/netshell.vxe' ...
Process '/romfs/netshell.vxe' (process Id = 0x800eff98) launched.
RTNET>
```

You must use the -i option with the rtp exec command because netshell reads from stdin in order to provide shell functionality. Do not detach from the kernel shell when starting **netshell.vxe**. That is, do not use an ampersand (&) at the end of the rtp exec command.

The shell's help command lists the available shell commands, which include blaster, blastee, echo, sockShow, ipShow, linkShow, fibShow, neighShow, and ping. To stop the application, press the ENTER key.

You can, for example, run linkShow as follows:

```
RTNET> linkShow

lo: <UP,BROADCAST,RUNNING>  ifindex 1  mtu 1500  state UP
        type loopback 00:00:00:00:00:00  brd ff:ff:ff:ff:ff:ff
                  bytes    packets     mcasts     errors
         RX:          0          0          0          0
         TX:          0          0          0          0
gei0: <UP,BROADCAST,RUNNING>  ifindex 2  mtu 1500  state UP
        type ethernet b8:ae:ed:7d:ab:30  brd ff:ff:ff:ff:ff:ff
                  bytes    packets     mcasts     errors
         RX:        256          3          1          0
         TX:        238          3          0          0
ret = 2
RTNET>
```

Related information

Kernel Demo Applications on page 40
Multicast Receive and Send Example on page 43

3. Multicast Receive and Send Example

This example code demonstrates multicast usage. It can be run from the kernel shell.

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <time.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int sendfd    = -1;
int receivefd = -1;
char               msgbuf[256] = {0};

void create_send_socket(void)
{
    if ((sendfd = socket (AF_INET, SOCK_DGRAM, 0)) < 0)
    {
        printf ("create send socket failed.\n");
    }
}
```

```
void free_send_socket(void)
{
    if (sendfd > 0)
    {
        close (sendfd);
        sendfd = -1;
    }
}

/* multicastAddr and outIfAddr are "XXX.XXX.XXX.XXX" */
int send_multicast_data(char* multicastAddr, unsigned short dstPort,
                        char* outIfAddr,
                        unsigned char ttl,
                        char* data, unsigned int dataLen)
{
    struct sockaddr_in mcaddr;
    struct sockaddr_in ifaddr;

    if (strlen(data) < dataLen)
    {
        printf ("gived data length is too big\n");
        return -1;
    }

    memset (&mcaddr,0,sizeof(mcaddr));
    mcaddr.sin_family      = AF_INET;
    mcaddr.sin_addr.s_addr = inet_addr(multicastAddr);
    mcaddr.sin_port        = htons(dstPort);

    memset (&ifaddr,0,sizeof(ifaddr));
    ifaddr.sin_family      = AF_INET;
    ifaddr.sin_addr.s_addr = inet_addr(outIfAddr);

    if (setsockopt(sendfd, IPPROTO_IP, IP_MULTICAST_TTL,
                   &ttl, sizeof (ttl)) < 0)
    {
        printf ("send socket set multicast ttl failed.\n");
        return -1;
    }

    if (setsockopt(sendfd, IPPROTO_IP, IP_MULTICAST_IF,
                   &ifaddr.sin_addr, sizeof (struct in_addr)))
    {
        printf ("send socket set multicast if failed.\n");
        return -1;
    }

    if (sendto(sendfd, data, dataLen, 0,
               (struct sockaddr *) &mcaddr, sizeof (mcaddr)) < 0)
    {
        printf ("send socket send packet failed.\n");
        return -1;
    }

    printf ("send socket send packet success(data:%s).\n", data);
    return 0;
}
```

```
void create_receive_socket (void)
{
    if ((receivefd = socket (AF_INET, SOCK_DGRAM, 0)) < 0)
    {
        printf ("create receive socket failed\n");
    }
}

void free_receive_socket (void)
{
    if (receivefd > 0)
    {
    close (receivefd);
    receivefd = -1;
    }
}

/* multicastAddr and inIfAddr are "XXX.XXX.XXX.XXX" */
int receive_multicast_data(char* multicastAddr, unsigned short recvPort,
                           char* inIfAddr)
{
    struct sockaddr_in addr;
    int                nbytes, addrlen;
    struct ip_mreq     mreq;
    u_int              yes = 1;

    if (setsockopt (receivefd, SOL_SOCKET, SO_REUSEADDR,
                    &yes, sizeof (yes)) < 0)
    {
        printf ("receive socket set reuse ADDR failed.\n");
        return -1;
    }

    memset (&addr, 0, sizeof(addr));
    addr.sin_len           = sizeof (addr);
    addr.sin_family        = AF_INET;
    addr.sin_addr.s_addr   = INADDR_ANY;
    addr.sin_port          = htons (recvPort);

    if (bind (receivefd, (struct sockaddr *) &addr, sizeof (addr)) < 0)
    {
        printf ("receive socket bind port failed.\n");
        return -1;
    }

    addr.sin_addr.s_addr   = inet_addr (inIfAddr);
    if (setsockopt(receivefd, IPPROTO_IP, IP_MULTICAST_IF,
                       &addr.sin_addr,  sizeof(struct in_addr)) < 0)
        {
            printf ("receive set multicast if failed.\n");
            return -1;
        }

    mreq.imr_multiaddr.s_addr = inet_addr (multicastAddr);
    mreq.imr_interface.s_addr = INADDR_ANY;
    /* if not set IP_MULTICAST_IF option, you could use below commented statement in here */
```

```
    /* mreq.imr_interface.s_addr = inet_addr (inIfAddr); */
    if (setsockopt(receivefd, IPPROTO_IP, IP_ADD_MEMBERSHIP,
                   &mreq, sizeof (mreq)) < 0)
    {
        printf ("receive socket join group failed.\n");
        return -1;
    }

    addrlen = sizeof (addr);
    if ((nbytes = recvfrom (receivefd, msgbuf, 256, 0,
                            (struct sockaddr *) &addr,
                            (socklen_t *)&addrlen)) < 0)
    {
        printf ("receive socket recv packet failed.\n");
        return -1;
    }
    printf ("receive socket receive packet success(data:%s).\n", msgbuf);
    if (setsockopt(receivefd, IPPROTO_IP, IP_DROP_MEMBERSHIP,
                        &mreq, sizeof (mreq)) < 0)
        {
            printf ("receive socket drop group failed.\n");
            return -1;
        }
    return 0;
}

/* multicastAddr and inIfAddr are "XXX.XXX.XXX.XXX" */
int receive_multicast_data_source(char* multicastAddr, char* multicastSrcAddr,
                                  unsigned short recvPort,
                            char* inIfAddr)
{
    struct sockaddr_in addr;
    int                nbytes, addrlen;
    struct ip_mreq_source     mreq_source;
    u_int              yes = 1;

    if (setsockopt (receivefd, SOL_SOCKET, SO_REUSEADDR,
                    &yes, sizeof (yes)) < 0)
    {
        printf ("receive socket set reuse ADDR failed.\n");
        return -1;
    }

    memset (&addr, 0, sizeof(addr));
    addr.sin_len           = sizeof (addr);
    addr.sin_family        = AF_INET;
    addr.sin_addr.s_addr   = INADDR_ANY;
    addr.sin_port          = htons (recvPort);

    if (bind (receivefd, (struct sockaddr *) &addr, sizeof (addr)) < 0)
    {
        printf ("receive socket bind port failed.\n");
        return -1;
    }

    addr.sin_addr.s_addr   = inet_addr (inIfAddr);
    if (setsockopt(receivefd, IPPROTO_IP, IP_MULTICAST_IF,
```

```
                        &addr.sin_addr,  sizeof(struct in_addr)) < 0)
        {
            printf ("receive set multicast if failed.\n");
            return -1;
        }

    mreq_source.imr_multiaddr.s_addr = inet_addr (multicastAddr);
    mreq_source.imr_sourceaddr.s_addr = inet_addr (multicastSrcAddr);
    mreq_source.imr_interface.s_addr = INADDR_ANY;
    /* if not set IP_MULTICAST_IF option, you could use below commented statement in here */
    /* mreq_source = inet_addr (inIfAddr); */
    if (setsockopt(receivefd, IPPROTO_IP, IP_ADD_SOURCE_MEMBERSHIP,
                   &mreq_source, sizeof (mreq_source)) < 0)
    {
        printf ("receive socket join source group failed.\n");
        return -1;
    }

    addrlen = sizeof (addr);
    if ((nbytes = recvfrom (receivefd, msgbuf, 256, 0,
                            (struct sockaddr *) &addr,
                            (socklen_t *)&addrlen)) < 0)
    {
        printf ("receive socket recv packet failed.\n");
        return -1;
    }
    printf ("receive socket receive packet success(data:%s).\n", msgbuf);
    if (setsockopt(receivefd, IPPROTO_IP, IP_DROP_SOURCE_MEMBERSHIP,
                        &mreq_source, sizeof (mreq_source)) < 0)
        {
            printf ("receive socket drop source group failed.\n");
            return -1;
        }
    return 0;
}

void multicast_receive_example_src (char * groupAddr, char * srcAddr, unsigned short recvPort, cha
r* recvIfAddr)
{
        create_receive_socket();
        (void)receive_multicast_data_source(groupAddr, srcAddr, recvPort, recvIfAddr);
        free_receive_socket();
}


void multicast_receive_example (char * groupAddr, unsigned short recvPort, char* recvIfAddr)
{
        create_receive_socket();
        (void)receive_multicast_data(groupAddr, recvPort, recvIfAddr);
        free_receive_socket();
}

void multicast_send_example (char* multicastAddr,
                                 unsigned short dstPort,
                             char* outIfAddr,
                             unsigned char ttl,
                             char* data,
```

```
                              unsigned int dataLen)
{
        create_send_socket();
        (void)send_multicast_data(multicastAddr, dstPort, outIfAddr, ttl, data, dataLen);
        free_send_socket();
}
```

Usage From Kernel Shell

The code can be implemented as a kernel application, downloaded (or linked), and run from the kernel shell. For example:

```
-> multicast_receive_example("225.0.0.1", 5000, "192.168.200.96")
```

```
->  multicast_send_example("225.0.0.1", 5000, "192.168.200.95", 8, "multicast data", 14)
```

Related information

Kernel Demo Applications on page 40
RTP Demo Applications on page 42

7. RTNET AND CERTIFICATION

1. Real-Time Network Stack and Certification

The real-time network stack can be used in systems that will be certified.

While the real-time network stack does not have certification evidence, it is designed to facilitate a certification project.

2. Configuring VxWorks With the RTNET in the Kernel for Certifiable System Development

Create a VSB based on the certification configuration option, and include the RTNET layer. Create a VIP based on the VSB and the certification configuration profile, and add **INCLUDE_RTNET**. Set the location of your configuration file with **RTNET_JSON_FILE**.

About This Task

Note: This workflow is intended to assist in adapting applications to a system that has been configured in the same manner as a certified system, which can facilitate the certification process. It is not designed for developing the certified system itself. This workflow can also be used for any system that will not be certified, but that would benefit from using the Safety Profile features with the constraints of the certification configuration of VxWorks.

Procedure

1. Create a VSB project based on the fsl_imx6 BSP, and the VSB certification configuration option.

   To select the option:
   - With wrtool, use the -cert SAFETY_CERT_ARMARCH7 option.
   - With Workbench, in the Project Setup dialog, select **a board support package and CERT configuration** from the **Based on** drop-down list.

2. Add the **RTNET** layer, for Real-Time Network stack support.
3. Build the VSB project.
4. Create a VIP project based on the VSB project, and the **PROFILE_SAFETY_CERT_RTP** profile.
5. Add the **INCLUDE_RTNET** component.
6. Set the **RTNET_JSON_FILE** configuration parameter to the filepath for your configuration file.

   The default assignment for the **RTNET_JSON_FILE** parameter is "**/romfs/rtnet.json**". See RTNET JSON Configuration File Reference on page 27.

7. If you want to store your configuration file in ROMFS, create a ROMFS project and add the file to it.

   Make the ROMFS project subordinate to your VIP project. For information about ROMFS projects, see the *VxWorks 7 Kernel and Application Projects Guide*.

8. Build the VIP project.

3. Configuring VxWorks With an RTP RTNET Stack Application for Certifiable System Development

Create a VSB based on the certification configuration option, and include the RTNET layer. Create a VIP based on the VSB, and the certification configuration profile. Create an RTP RTNET stack application, and store it in ROMFS.

About This Task

Note: This workflow is intended to assist in adapting applications to a system that has been configured in the same manner as a certified system, which can facilitate the certification process. It is not designed for developing the certified system itself. This workflow can also be used for any system that will not be certified, but that would benefit from using the Safety Profile features with the constraints of the certification configuration of VxWorks.

Procedure

1. Create a VSB project based on the fsl_imx6 BSP, and the VSB certification configuration option.

   To select the option:
   - With wrtool, use the -cert SAFETY_CERT_ARMARCH7 option.
   - With Workbench, in the Project Setup dialog, select **a board support package and CERT configuration** from the **Based on** drop-down list.

2. Add the **RTNET** layer, for Real-Time Network stack support.
3. Build the VSB project.
4. Create a VIP project based on the VSB project, and the **PROFILE_SAFETY_CERT_RTP** profile.
5. Configure the RTP resource access control and RTP time partition scheduling features that are provided by the Safety Profile product.

   See the *VxWorks 7 Programmer's Guide: Safety Profile Facilities for RTP Application Control*.

6. Create and build an RTP project for RTNET.

   See VxWorks RTNET Configuration Reference on page 24.

7. Create a ROMFS project for the RTP RTNET application.

   Create a ROMFS project that is subordinate to your VIP project, and add your RTP RTNET application executable (**.vxe**) to it. See the *VxWorks 7 Kernel and Application Projects Guide*.

8. Configure VxWorks for starting the RTP application automatically, and write a startup script to start the stack automatically, as well as to to disable memory allocation.

   See Automatic Stack Startup and Disabling Memory Allocation in RTPs on page 50.

9. Build the VIP project.

4. Automatic Stack Startup and Disabling Memory Allocation in RTPs

The RTNET stack must be initiated programatically if the stack is going to be used in a certified system. In addition, memory allocation must be disabled after system initialization. For testing and demo purposes, however, the stack can be started automatically.

WNDRVR

For automatic start-up, configure the VIP project with the **INCLUDE_RTNET_SAFETY_CERT_RTP** component, and set its two parameters: **RTNET_RTP_EXEC_PATH** and **RTNET_JSON_FILE**.

The **INCLUDE_RTNET_SAFETY_CERT_RTP** component configures the time partition scheduler, sets up object access control, and starts RTNET at boot time. For details about the component's configuration of the scheduler and access control, see *installDir* **/ vxworks-7/pkgs_v2/net/rtnet/configlette/usrRtnetSafetyCert.c**.

For certification purposes, memory allocation must be disabled. To do so, call memAllocDisable( ) in your startup application at the point when all initialization has been completed (that is, after RTP start-up).